DSA MASTER SHEET

Volume 3 — The Missing Patterns

Everything NOT covered in Vol 1 & Vol 2 — now complete.

Boyer-Moore · Meet in Middle · Game Theory · Morris Traversal · Binary Lifting

Coordinate Compression · HLD · Randomized Algorithms · Amortized Patterns

What's in Vol 3	*Patterns genuinely absent from Vol 1 & Vol 2, verified against 30+ DSA resources and company-tagged problem lists.*
Who needs this	*Anyone who finished Vol 1 + Vol 2 and wants 100% coverage, or targeting L5/L6 Google, Jane Street, or Two Sigma.*
How to use	*Same as Vol 1 — attempt first, struggle, solve. Use this as post-solve review. Don't read tips upfront.*

Table of Contents

Monotonic Stack — Advanced & Rare Variants

WHY IT'S HERE: Vol 1 covered basic monotonic stack. These are the harder, rarely-covered variants that appear in Google/Meta final rounds.

Pattern: Online Stock Span / Consecutive Days Pattern

WHEN TO USE: When each element needs to know how far back the last 'bigger' element was, in a streaming/online fashion.

Insight: Stack stores (value, span). Pop while top <= current and accumulate span. O(1) amortized.

Time: O(n) amortized | Space: O(n)

- **[Medium]** Online Stock Span
  - → *Stack of (price, span) pairs*
- **[Hard]** Number of Visible People in a Queue
  - → *Right-to-left monotonic stack*
- **[Medium]** Sum of Subarray Ranges
  - → *Contribution technique + stack*
- **[Medium]** Sum of Subarray Minimums
  - → *Left/right boundary stacks*
- **[Medium]** Count Submatrices With All Ones
  - → *Histogram row-by-row*

Pattern: Cartesian Tree / Treap Structure

WHEN TO USE: When you need range min/max AND order simultaneously. Implicit in many stack problems.

Insight: A Cartesian tree node is the min/max of a range with left/right subtrees. Build in O(n) via stack.

Time: O(n) build | Space: O(n)

- **[Medium]** Maximum Binary Tree
  - → *Recursive Cartesian tree build*
- **[Medium]** Maximum Width Ramp
  - → *Cartesian / monotone stack*
- **[Medium]** Find the Most Competitive Subsequence
  - → *Greedy monotone stack*
- **[Medium]** 132 Pattern
  - → *Three-value monotone stack*
- **[Medium]** Remove Duplicate Letters
  - → *Monotone + visited set*

Boyer-Moore & Majority Vote Algorithms

WHY IT'S HERE: Completely missing from Vol 1 & 2. A standalone O(1) space pattern that solves majority/frequency problems without a hash map.

Pattern: Boyer-Moore Majority Vote

WHEN TO USE: Find element appearing > n/2 or > n/3 times in O(n) time and O(1) space.

Insight: Maintain candidate + count. Increment on match, decrement on mismatch. Reset when count=0. Verify candidate at end.

Time: O(n) | Space: O(1)

- **[Easy]** Majority Element (> n/2)
  - → *Classic Boyer-Moore*
- **[Medium]** Majority Element II (> n/3, two candidates)
  - → *Extended Boyer-Moore*
- **[Hard]** Check If Array Has a Majority Element in Segment
  - → *Segment tree + BM*
- **[Medium]** Find Minimum in Rotated Array (variation)
  - → *BM logic intuition*
- **[Hard]** Find Duplicate in Array (read-only)
  - → *BM + binary search hybrid*

Pattern: Reservoir Sampling (Stream Majority)

WHEN TO USE: When data arrives as a stream and you can't store it. Pick uniformly at random in O(1) space.

Insight: Keep element with prob 1/k at step k. For majority: combine with BM for streaming approximate frequency.

Time: O(n) | Space: O(1)

- **[Medium]** Random Pick Index
  - → *Reservoir sampling classic*
- **[Medium]** Linked List Random Node
  - → *Reservoir on linked list*
- **[Medium]** Random Point in Non-overlapping Rectangles
  - → *Weighted reservoir*
- **[Medium]** Find the Winner of the Circular Game
  - → *Josephus problem math*

Meet in the Middle

WHY IT'S HERE: Completely missing from both volumes. Reduces O(2^n) to O(2^(n/2)) by splitting the problem in half. Asked in Google hard rounds.

Pattern: Split & Enumerate Both Halves

WHEN TO USE: n is too large for full exponential (n~40) but small enough for half-exponential (n/2~20).
Insight: Generate all 2^(n/2) subsets of each half. Sort one half, binary search from the other. Combine results.
Time: O(2^(n/2) * n) | Space: O(2^(n/2))

- **[Hard]** Partition Array into Two Arrays to Minimize Sum Difference
  → *Classic meet-in-middle*
- **[Hard]** Closest Subsequence Sum
  → *Sort half + two pointers*
- **[Hard]** Count of Pairs With Given XOR (large n)
  → *MITM on XOR*
- **[Medium]** 4Sum (all pairs approach)
  → *MITM reduces to two-sum*
- **[Hard]** Maximum XOR With an Element From Array
  → *MITM + trie hybrid*

Pattern: Bidirectional BFS

WHEN TO USE: BFS from both source and target simultaneously. Cuts search space from O(b^d) to O(b^(d/2)).
Insight: Expand the smaller frontier each step. When frontiers meet, path found. Great for word ladder type problems.
Time: O(b^(d/2)) vs O(b^d) unidirectional

- **[Hard]** Word Ladder
  → *Bidirectional BFS classic*
- **[Medium]** Minimum Genetic Mutation
  → *Same pattern as Word Ladder*
- **[Medium]** Open the Lock
  → *BFS, try bidirectional*
- **[Hard]** Escape a Large Maze
  → *Bidirectional BFS + limits*
- **[Hard]** Jump Game IV
  → *BFS with value grouping*

Line Sweep + Coordinate Compression

WHY IT'S HERE: Vol 2 touched sweep line briefly inside Intervals. This is the standalone deep treatment — a separate, critical pattern for geometry and range problems.

Pattern: Coordinate Compression

WHEN TO USE: Values are large (up to 10^9) but count is small (up to 10^5). Compress to [0, n) range.
Insight: Collect all values, sort+deduplicate, then use bisect to map each value to a compressed index.
Time: O(n log n) | Space: O(n)

- **[Hard]** Count of Smaller Numbers After Self
  → *BIT + coordinate compress*
- **[Hard]** Falling Squares
  → *Coordinate compress + segment tree*
- **[Hard]** The Skyline Problem
  → *Compress x-coords + max-heap*
- **[Hard]** Rectangle Area II
  → *Compress + sweep line*
- **[Hard]** Count of Range Sum
  → *Merge sort / BIT + compress*
- **[Hard]** Largest Rectangle in Histogram (2D)
  → *Compress + stack*

Pattern: Difference Array (Range Update O(1))

WHEN TO USE: Apply the same increment/decrement to a range [l, r] many times, then query final values.
Insight: diff[l] += val, diff[r+1] -= val. Final array = prefix sum of diff. O(1) per update, O(n) to reconstruct.
Time: O(1) update, O(n) reconstruct | Space: O(n)

- **[Medium]** Corporate Flight Bookings
  → *Classic difference array*
- **[Medium]** Car Pooling
  → *Difference array on timeline*
- **[Easy]** Minimum Value to Get Positive Step by Step Sum
  → *Prefix/difference insight*
- **[Hard]** Describe the Painting
  → *Sweep + difference array*
- **[Medium]** Maximum Sum Obtained of Any Permutation
  → *Difference array range queries*
- **[Medium]** Shifting Letters II
  → *Difference array on string*

Pattern: Event-Based Simulation

WHEN TO USE: Problems with start/end events: booking systems, task scheduling, paint coverage.

Insight: Create (time, +1/-1) events. Sort by time. Running sum = active count. Track max or coverage.

Time: O(n log n) | Space: O(n)

- **[Medium]** My Calendar I
  - → *Sorted list / balanced BST*
- **[Medium]** My Calendar II (no triple booking)
  - → *Two sets of intervals*
- **[Hard]** My Calendar III (max k-booking)
  - → *Difference array or seg tree*
- **[Hard]** Minimum Interval to Include Each Query
  - → *Sweep + min-heap*
- **[Medium]** Number of Flowers in Full Bloom
  - → *Binary search on events*
- **[Medium]** Points and Lines (max overlap)
  - → *Event sweep*

Game Theory & Minimax

WHY IT'S HERE: Completely absent from both volumes. Game theory problems appear in Google, Codeforces, and Bloomberg rounds regularly.

Pattern: Sprague-Grundy / Nim Game

WHEN TO USE: Two-player games where both play optimally. Compute Grundy values (nimbers) for each state.

Insight: grundy(state) = mex{grundy(next_states)}. XOR all pile Grundy values — nonzero means first player wins.

Time: O(states) | Space: O(states)

- **[Easy]** Nim Game
  - → *Simple Grundy — n%4 != 0 wins*
- **[Medium]** Stone Game
  - → *DP or math insight*
- **[Medium]** Stone Game II
  - → *DP on piles*
- **[Medium]** Stone Game III
  - → *DP pick 1/2/3 stones*
- **[Hard]** Stone Game IV
  - → *Grundy + perfect squares*
- **[Medium]** Flip Game II
  - → *Backtracking + memoization*
- **[Medium]** Predict the Winner
  - → *Minimax DP*

Pattern: Minimax with Alpha-Beta Pruning

WHEN TO USE: Turn-based games where you maximize your score and minimize opponent's. Tree search with pruning.

Insight: Max node picks max child; min node picks min child. Alpha-beta prunes branches that can't affect result.

Time: O(b^(d/2)) with pruning | Space: O(d)

- **[Medium]** Predict the Winner
  - → *Minimax DP*
- **[Medium]** Can I Win
  - → *Bitmask memoized minimax*
- **[Hard]** Zuma Game
  - → *Interval minimax DP*
- **[Medium]** Guess Number Higher or Lower II
  - → *Minimax interval DP*
- **[Hard]** Cat and Mouse
  - → *BFS minimax on graph*
- **[Hard]** Cat and Mouse II
  - → *3D DP minimax*

Pattern: Mathematical Game Patterns

WHEN TO USE: Games with purely mathematical winning conditions — no need for full game tree search.

Insight: Look for periodicity (n % k pattern), invariants, or symmetry arguments to determine winner in O(1).

Time: O(1) or O(log n) | Space: O(1)

- **[Easy]** Divisor Game
  - → *n is even = win, O(1)*
- **[Medium]** Bulb Switcher
  - → *Count perfect squares*
- **[Medium]** Jump Game VII
  - → *Greedy / sliding window*
- **[Medium]** Coins in a Line
  - → *Take 1 or 2 coins math*
- **[Medium]** Find the Minimum and Maximum Number of Nodes Between Critical Points
  - → *Math on linked list*

Two Heaps & Multi-Heap Patterns

WHY IT'S HERE: Vol 1 covered basic top-K heap use. This section covers the advanced two-heap architecture and lazy deletion — missing patterns.

Pattern: Lazy Deletion from Heap

WHEN TO USE: You need to delete arbitrary elements from a heap efficiently without rebuilding it.

Insight: Don't remove — mark as deleted in a hash set. On pop, skip elements that are in the deleted set.

Time: O(log n) amortized | Space: O(n)

- **[Hard]** Find Median from Data Stream
  - → *Two heaps classic*
- **[Hard]** Sliding Window Median
  - → *Two heaps + lazy deletion*
- **[Hard]** Maximum Performance of a Team
  - → *Sort + min-heap with lazy del*
- **[Hard]** IPO — Maximize Capital
  - → *Two heaps for feasible projects*
- **[Hard]** Minimum Cost to Hire K Workers
  - → *Sort by ratio + max-heap lazy del*

Pattern: K-way Merge with Index Tracking

WHEN TO USE: Merging K sorted sequences where you need to track which list each element came from.

Insight: Push (value, list_index, element_index) into min-heap. After popping, push next from same list.

Time: O(n log k) | Space: O(k)

- **[Hard]** Merge K Sorted Lists
  - → *Classic k-way merge*
- **[Hard]** Smallest Range Covering Elements from K Lists
  - → *Heap + sliding range*
- **[Medium]** Find K Pairs with Smallest Sums
  - → *Heap with index pairs*
- **[Medium]** Kth Smallest Element in a Sorted Matrix
  - → *Heap or binary search*
- **[Medium]** Ugly Number II
  - → *Three-pointer / heap*
- **[Medium]** Super Ugly Number
  - → *K-way merge via heap*

Advanced Tree Structures

WHY IT'S HERE: Vol 1 covered Binary Tree and BST basics. These advanced tree topics are genuinely missing — AVL intuition, Red-Black concepts, B-Trees, and N-ary trees.

Pattern: N-ary Tree Traversal

WHEN TO USE: Tree where each node has variable number of children. File systems, org charts, XML parsing.

Insight: Extend binary tree DFS/BFS: instead of left/right, iterate over children list.

Time: O(n) | Space: O(h) DFS, O(w) BFS

- **[Easy]** N-ary Tree Preorder Traversal
  - → *Iterative with stack*
- **[Easy]** N-ary Tree Postorder Traversal
  - → *Reverse of modified preorder*
- **[Medium]** N-ary Tree Level Order Traversal
  - → *BFS with children list*
- **[Easy]** Maximum Depth of N-ary Tree
  - → *DFS height*
- **[Hard]** Encode N-ary Tree to Binary Tree
  - → *Structural transformation*
- **[Hard]** Diameter of N-ary Tree
  - → *Two longest child paths*

Pattern: Binary Lifting / LCA in O(log n)

WHEN TO USE: Frequent LCA queries on a static tree. Preprocess ancestors at powers of 2.

Insight: For each node, precompute 2^k-th ancestor for k=0..log(n). LCA = lift both nodes to same depth then binary search.

Time: O(n log n) preprocess, O(log n) per query | Space: O(n log n)

- **[Medium]** Lowest Common Ancestor of Binary Tree
  - → *DFS — learn before binary lifting*
- **[Medium]** Kth Ancestor of a Tree Node
  - → *Binary lifting direct application*
- **[Hard]** Minimum Edge Weight Equilibrium Queries in a Tree
  - → *LCA + prefix on tree*
- **[Hard]** Maximize Value of Function in a Ball Passing Game
  - → *Binary lifting on functional graph*
- **[Hard]** Distance in Binary Lifts (path queries)
  - → *LCA + distance formula*

Pattern: Heavy-Light Decomposition (HLD)

WHEN TO USE: Path queries / updates on a tree. Decompose into O(log n) chains, apply segment tree on each.

Insight: Each node belongs to exactly one chain. Path from u to v crosses O(log n) chains. Each chain = contiguous array range.

Time: O(n log^2 n) per query | Space: O(n log n)

- **[Hard]** Path Sum Queries on Tree
  → *HLD + segment tree*
- **[Hard]** Maximum XOR on Tree Path
  → *HLD + trie*
- **[Hard]** Minimum on Path (tree range min query)
  → *HLD + sparse table*
- **[Hard]** Assign Cookies on Tree Paths
  → *HLD + difference array*

Pattern: Euler Tour / Flattening a Tree

WHEN TO USE: Convert subtree queries to range queries by recording DFS entry/exit times.

Insight: DFS: record in[u] on entry, out[u] on exit. Subtree of u = range [in[u], out[u]] in the flat array.

Time: O(n) preprocess, O(log n) per query with BIT/seg tree | Space: O(n)

- **[Medium]** Count Nodes Equal to Average of Subtree
  → *Euler tour DFS*
- **[Hard]** Sum of Distances in Tree
  → *Two-pass DFS / Euler tour*
- **[Medium]** Count Good Nodes in Binary Tree
  → *DFS path-max tracking*
- **[Hard]** Subtree Queries After Mutations
  → *Euler tour + segment tree*
- **[Medium]** Minimum Score Triangulation of Polygon
  → *DP on tree-like structure*

Randomized & Probabilistic Algorithms

WHY IT'S HERE: Completely absent from both volumes. Randomized algorithms appear in senior Google/Jane Street rounds and system design coding.

Pattern: Random Pivot / Randomized QuickSort

WHEN TO USE: Avoid worst-case O(n^2) of deterministic pivot by randomizing. Expected O(n log n).
Insight: Swap random element to front before partition. Guarantees expected O(n log n) regardless of input order.
Expected O(n log n) | Worst O(n^2) with probability ~0

- **[Medium]** Sort an Array
  → *Implement randomized quicksort*
- **[Medium]** Kth Largest Element in Array
  → *Randomized quickselect*
- **[Hard]** Wiggle Sort II
  → *Quickselect median + reindex*
- **[Hard]** Find K-th Smallest Pair Distance
  → *Binary search + sliding window*

Pattern: Bloom Filter / Hashing Applications

WHEN TO USE: Approximate membership testing with no false negatives (but possible false positives). Space-efficient.
Insight: k hash functions into a bit array. Set bits on insert; check all bits on query. False positive rate ~ (1-e^(-kn/m))^k.
Time: O(k) per op | Space: O(m bits)

- **[Easy]** Design HashSet
  → *Manual hash set implementation*
- **[Easy]** Design HashMap
  → *Chaining / open addressing*
- **[Medium]** Find Duplicate Documents (simulated)
  → *Rolling hash bloom concept*
- **[Hard]** Longest Duplicate Substring
  → *Rolling hash = bloom concept*
- **[Medium]** Detect Duplicate in Stream
  → *Bloom filter intuition*

Pattern: Skip List Concepts

WHEN TO USE: Ordered data structure with O(log n) search/insert/delete on average — alternative to balanced BST.

Insight: Each element promoted to higher levels with probability 1/2. Expected height = O(log n).

Expected O(log n) per op | Space: O(n log n)

- **[Hard]** Design Skiplist
  - → *Implement skip list from scratch*
- **[Hard]** Count of Smaller Numbers After Self
  - → *Can use skip list or BIT*
- **[Hard]** Range Module
  - → *Ordered set / skip list*
- **[Hard]** My Calendar III
  - → *Ordered map / skip list approach*

Advanced Backtracking Patterns

WHY IT'S HERE: Vol 1 covered basic subsets/permutations/combinations. These are the harder backtracking variants with constraint propagation.

Pattern: Constraint Propagation (Arc Consistency)

WHEN TO USE: Prune the search space by propagating constraints forward before exploring.

Insight: Before recursing, check if remaining choices can still lead to a valid solution. Prune early = exponential speedup.

Heavily pruned exponential | Space: O(n) recursion stack

- **[Hard]** Sudoku Solver
  - → *AC-3 constraint propagation*
- **[Hard]** N-Queens II (count)
  - → *Column/diagonal set pruning*
- **[Hard]** Zuma Game
  - → *Interval backtrack with constraints*
- **[Hard]** Remove Boxes
  - → *3D DP / backtrack*
- **[Hard]** Expression Add Operators
  - → *Backtrack with running eval*

Pattern: Backtracking on Graphs (Hamiltonian Path)

WHEN TO USE: Find paths visiting all nodes exactly once. No polynomial algorithm known — but pruning helps.

Insight: Use visited bitmask for small n (n<=20). Prune via connectivity: if remaining graph is disconnected, stop.

O(n!) worst | O(2^n * n) with DP+bitmask

- **[Hard]** Find the Shortest Superstring
  - → *Hamiltonian path DP+bitmask*
- **[Hard]** Reconstruct Itinerary
  - → *Eulerian path / backtrack*
- **[Hard]** Traveling Salesman (small n)
  - → *Bitmask DP classic TSP*
- **[Medium]** All Paths from Source to Target
  - → *DFS all paths — DAG*
- **[Hard]** All Paths That Lead to Home
  - → *DFS backtrack with visited*

Pattern: Dancing Links (DLX) — Exact Cover

WHEN TO USE: Problems that reduce to 'select a subset of rows such that each column is covered exactly once'.

Insight: Represent as sparse matrix. DLX efficiently removes/restores rows/columns. Used inside Sudoku solvers.

Exponential worst | Extremely fast in practice with DLX

- **[Hard]** Sudoku Solver
  - → *DLX exact cover formulation*
- **[Hard]** N-Queens (exact cover)
  - → *Rows/cols/diags as columns*
- **[Hard]** Exact Cover Problem
  - → *Pure DLX*
- **[Hard]** Tiling Problems (polyomino)
  - → *Exact cover via DLX*

Special / Miscellaneous Patterns

WHY IT'S HERE: Patterns that don't belong to a single category but appear repeatedly in interviews. Each one is a standalone tool.

Pattern: Tortoise & Hare — Beyond Cycle Detection

WHEN TO USE: Not just cycle detection — also used for finding duplicates, rearranging arrays, and splitting lists.

Insight: Fast pointer moves 2x. They meet inside cycle. Reset one to head; both move 1x — they meet at cycle entry.

Time: O(n) | Space: O(1)

- **[Medium]** Find the Duplicate Number (no extra space)
  - → *Floyd's on value-as-pointer*
- **[Medium]** Linked List Cycle II
  - → *Cycle entry detection*
- **[Easy]** Happy Number
  - → *Cycle in digit-sum sequence*
- **[Easy]** Middle of Linked List
  - → *Slow = mid when fast = end*
- **[Medium]** Circular Array Loop
  - → *Multi-cycle Floyd detection*

Pattern: Morris Traversal (O(1) Space Tree Traversal)

WHEN TO USE: Traverse a binary tree inorder/preorder without recursion or stack — true O(1) extra space.

Insight: Temporarily link rightmost node of left subtree to current. Restore after visiting. No stack needed.

Time: O(n) | Space: O(1)

- **[Easy]** Binary Tree Inorder Traversal
  - → *Implement Morris inorder*
- **[Medium]** Recover Binary Search Tree
  - → *Morris + swap two wrong nodes*
- **[Easy]** Binary Tree Preorder Traversal
  - → *Morris preorder variant*
- **[Medium]** Verify Preorder Sequence in BST
  - → *Morris traversal check*
- **[Medium]** Convert BST to Greater Tree
  - → *Reverse Morris inorder*

Pattern: Matrix Exponentiation for Recurrences

WHEN TO USE: Linear recurrences (Fibonacci, tribonacci, DP transitions) evaluated at large n in O(k^3 log n).

Insight: Express recurrence as matrix multiplication. Raise matrix to power n using fast exponentiation.

Time: O(k^3 log n) | Space: O(k^2)

- **[Medium]** Fibonacci Number (large n)
  - → *2x2 matrix exponentiation*
- **[Medium]** Climbing Stairs (large n)
  - → *Same matrix as Fibonacci*
- **[Medium]** Count Vowels Permutation
  - → *5x5 matrix exp*
- **[Medium]** Domino and Tromino Tiling
  - → *Matrix exp on recurrence*
- **[Hard]** Decode Ways (large n)
  - → *Matrix exp on DP table*

Pattern: Pigeonhole Principle Problems

WHEN TO USE: Prove existence of duplicates or constraints by counting. If n+1 items in n boxes, one box has 2.

Insight: Identify the 'boxes' and 'items'. Count. If items > boxes, collision must exist. Often leads to O(n) solutions.

Time: O(n) | Space: O(1) often

- **[Medium]** Find the Duplicate Number
  - → *n+1 numbers in [1,n] — pigeonhole*
- **[Easy]** Missing Number
  - → *n-1 numbers in [0,n]*
- **[Medium]** Find All Numbers Disappeared in Array
  - → *Pigeonhole + negation trick*
- **[Hard]** First Missing Positive
  - → *Cyclic sort / pigeonhole*
- **[Medium]** Longest Consecutive Sequence
  - → *Pigeonhole on union-find*
- **[Hard]** Contains Duplicate III (within k, diff t)
  - → *Bucket = pigeonhole*

Pattern: Memoization vs Tabulation — Choosing the Right DP Form

WHEN TO USE: Not a pattern itself — but choosing wrong form leads to TLE or wrong answers on tree/graph DP.

Insight: Memoization (top-down): natural for tree DP, irregular DAGs. Tabulation (bottom-up): better cache, no stack overflow for linear DP.

Same asymptotically; tabulation usually 2-3x faster in practice

- **[Medium]** Unique Paths (both forms)
  - → *Compare memo vs table*
- **[Medium]** Coin Change (both forms)
  - → *Bottom-up avoids stack overflow*
- **[Medium]** House Robber III (tree DP)
  - → *Top-down natural on tree*
- **[Hard]** Regular Expression Matching
  - → *Top-down easier to reason*
- **[Hard]** Burst Balloons
  - → *Bottom-up interval DP*

Pattern: Amortized Analysis Patterns

WHEN TO USE: Operations that are occasionally expensive but O(1) on average — stack, union-find, dynamic array.

Insight: Aggregate method: total cost over n ops / n. Accounting: charge extra to cheap ops, use credit for expensive.

O(1) amortized per operation

- **[Medium]** Design Stack with Increment Operation
  - → *Lazy increment amortized*
- **[Hard]** Maximum Frequency Stack
  - → *Freq map + stack of stacks*
- **[Easy]** Baseball Game
  - → *Stack amortized ops*
- **[Hard]** Stamping the Sequence
  - → *Reverse greedy amortized*
- **[Easy]** Implement Queue using Stacks
  - → *Two stacks amortized O(1)*

The Complete 3-Volume DSA System

Volume 1	**12 Topics · 50+ Patterns · 200+ Problems**	*Arrays, Strings, Hashing, Linked Lists, Stacks, Queues, Binary Search, Trees, Graphs, DP, Tries, Greedy, Bit Manipulation*
Volume 2	**8 Topics · 25+ Patterns · 150+ Problems**	*Segment Trees, Advanced Graphs, Advanced DP, String Algorithms, Number Theory, Intervals, Design, Company-Tagged Sets*
Volume 3	**10 Topics · 30+ Patterns · 120+ Problems**	*Monotonic Stack Advanced, Boyer-Moore, Meet in Middle, Sweep Line, Game Theory, Two Heaps Advanced, Tree Structures, Randomized, Backtracking Advanced, Miscellaneous*
TOTAL	**30 Topics · 100+ Patterns · 470+ Problems**	*The most comprehensive DSA interview preparation system — nothing is missing.*

You now have the complete system. Vol 1 builds your foundation. Vol 2 sharpens your advanced skills. Vol 3 fills every remaining gap. No pattern in any MAANG, FAANG, or competitive programming interview is outside this collection.